文件读写

文件读写
例程实验效果简介
代码讲解
关于文件读写操作

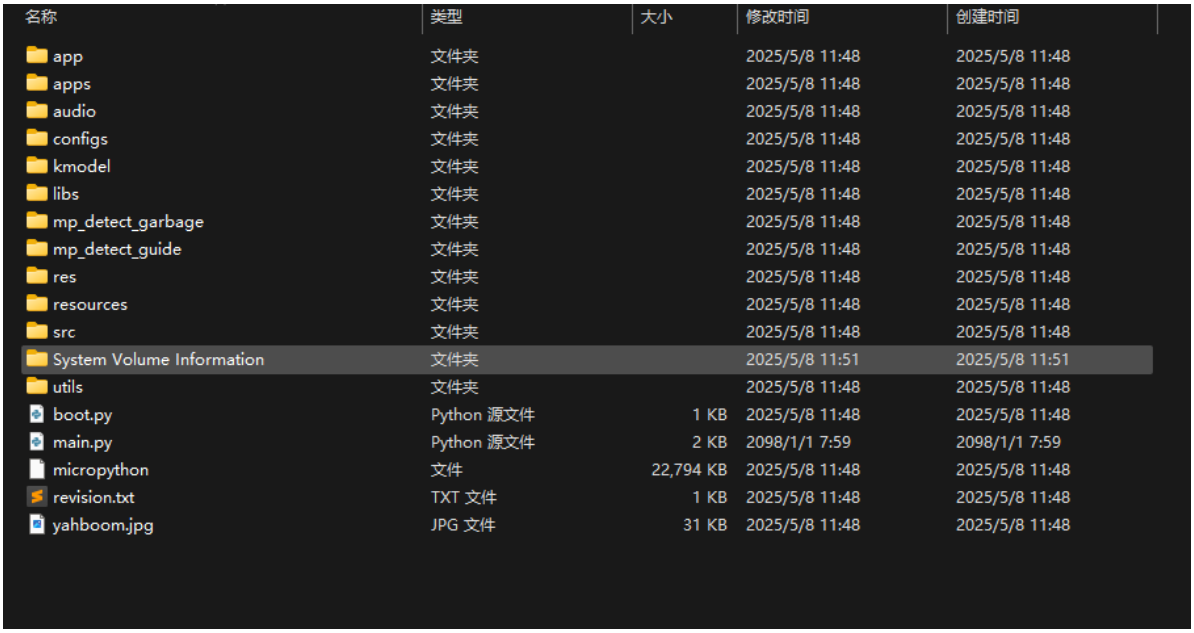
例程实验效果简介

> 我们经常会遇到需要将数据断电保存的情况，比如将传感器获取到的数据以文件的形式存储下来，
>
> 方便后续对数据的进一步操作。其实不管是在代码中用import导入模块，还是后面我们会学习到的“拍照保存图片”、“读取模型文件”
>
> 这些行为的底层都依赖于文件系统的读写操作。在本节教程中，我们将介绍一种比较常用的文件读写操作方法

本节例程代码位于：【源码汇总 / 02.Basic / 14.file.py】

我们用CanMV IDE打开例程代码，将K230用usb线连接到电脑上

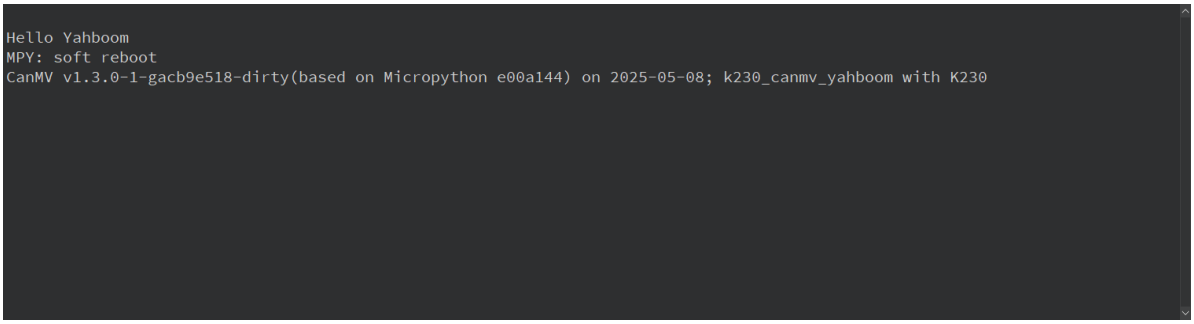
在系统的文件管理器中，打开CanMV对应的目录。我们进入 CanMV/sdcard这个目录里面



名称	类型	大小	修改时间	创建时间
app	文件夹		2025/5/8 11:48	2025/5/8 11:48
apps	文件夹		2025/5/8 11:48	2025/5/8 11:48
audio	文件夹		2025/5/8 11:48	2025/5/8 11:48
configs	文件夹		2025/5/8 11:48	2025/5/8 11:48
kmodel	文件夹		2025/5/8 11:48	2025/5/8 11:48
libs	文件夹		2025/5/8 11:48	2025/5/8 11:48
mp_detect_garbage	文件夹		2025/5/8 11:48	2025/5/8 11:48
mp_detect_guide	文件夹		2025/5/8 11:48	2025/5/8 11:48
res	文件夹		2025/5/8 11:48	2025/5/8 11:48
resources	文件夹		2025/5/8 11:48	2025/5/8 11:48
src	文件夹		2025/5/8 11:48	2025/5/8 11:48
System Volume Information	文件夹		2025/5/8 11:51	2025/5/8 11:51
utils	文件夹		2025/5/8 11:48	2025/5/8 11:48
boot.py	Python 源文件	1 KB	2025/5/8 11:48	2025/5/8 11:48
main.py	Python 源文件	2 KB	2098/1/1 7:59	2098/1/1 7:59
micropython	文件	22,794 KB	2025/5/8 11:48	2025/5/8 11:48
revision.txt	TXT 文件	1 KB	2025/5/8 11:48	2025/5/8 11:48
yahboom.jpg	JPG 文件	31 KB	2025/5/8 11:48	2025/5/8 11:48

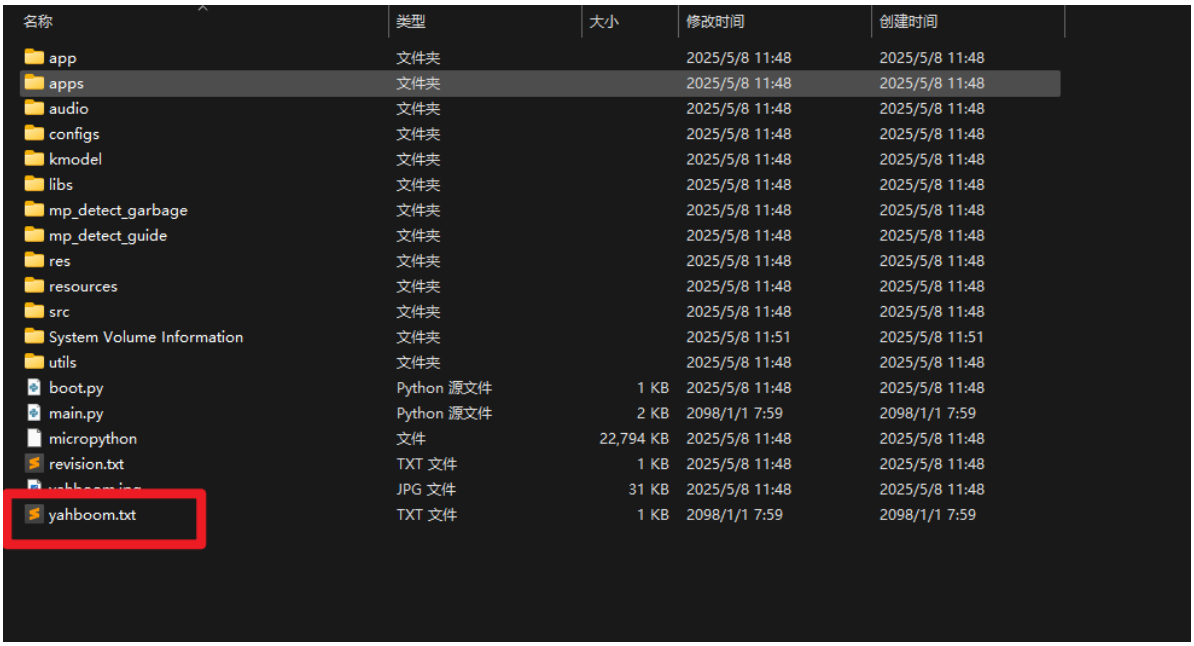
> CanMV/sdcard/ 目录对应在代码里面就是 /sdcard/

点击CanMV IDE左下角的运行按钮，同时打开下面的串行终端，可以看到这里输出了Hello Yahboom

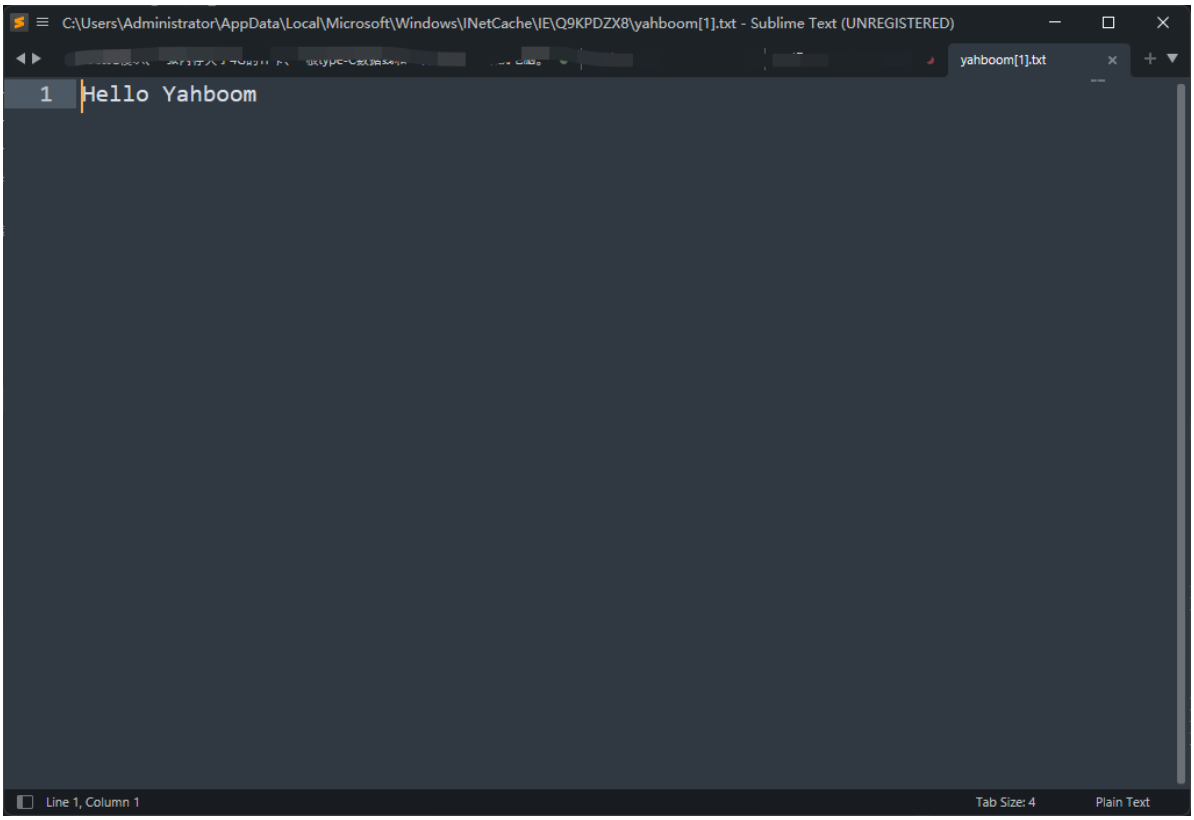


```
Hello Yahboom
MPY: soft reboot
CanMV v1.3.0-1-gacb9e518-dirty(based on Micropython e00a144) on 2025-05-08; k230_canmv_yahboom with K230
```

等待程序运行结束后，我们再回到文件管理器里面，可以看到这里多出来了一个yahboom.txt文件



我们双击打开这个文件，里面的内容是“Hello Yahboom”



代码讲解

```
# 文件写入
with open('/sdcard/yahboom.txt', 'w') as f:
    f.write("Hello Yahboom")

# 文件读取
with open('/sdcard/yahboom.txt', 'r') as f:
    print(f.read())
```

我们使用with open as 语法来打开文件系统中的文件

open()方法有两个参数，第一个参数是文件路径，第二个参数是打开模式，

常用的有 r (read，读取模式) 和 w (write, 写入模式)

> with open as .... 是一种更为推荐的文件操作方法
>
> 1. `with open as` 会自动关闭文件，无论代码是否发生异常。退出 with 代码块时，Python 会自动调用文件对象的 `close()` 方法。
> 2. 直接使用 `open()` 需要手动调用 `close()` 来关闭文件。如果忘记关闭或代码抛出异常，可能会造成资源泄露。
>
> 我们可以看一下用与不用的区别
>
> ```
> # with open as (推荐使用)
> with open('file.txt', 'r') as f:
>     content = f.read()
> # 文件自动关闭
>
> # 直接使用 open
> f = open('file.txt', 'r')
> content = f.read()
> f.close()  # 需要手动关闭
> ```

关于文件读写操作

MicroPython 中的文件操作与标准 Python 类似，但功能相对简化。

> 需要注意的限制：
>
> 1. 内存限制：避免一次性读取过大的文件
> 2. 文件系统：某些开发板可能只支持有限的文件系统功能
> 3. 路径操作：建议使用简单的相对路径

以下是主要的文件操作方法：

1. 打开文件

```
# 读取模式打开
f = open('test.txt', 'r')

# 写入模式打开
f = open('test.txt', 'w')

# 追加模式打开
f = open('test.txt', 'a')

# 二进制模式
f = open('test.txt', 'rb')
```

2. 基本读写操作

```
# 读取操作
f = open('test.txt', 'r')
content = f.read()      # 读取全部内容
line = f.readline()     # 读取一行
lines = f.readlines()   # 读取所有行到列表
f.close()

# 写入操作
f = open('test.txt', 'w')
f.write('hello\n')      # 写入字符串
f.writelines(['line1\n', 'line2\n'])  # 写入多行
f.close()
```

3. 推荐使用 with 语句

```
with open('test.txt', 'r') as f:
    content = f.read()
```

4. 文件系统操作（补充，在Micropython中不一定完全兼容）

```
import os

# 列出当前目录文件
os.listdir()

# 检查文件是否存在
os.stat('test.txt')

# 删除文件
os.remove('test.txt')

# 重命名文件
os.rename('old.txt', 'new.txt')
```